

Contents

Introduction	2
This course is for	2
Starting & Course Outline	3
Languages and Tools of Web Development	3
Front End Roadmap	4
How the web works.....	5
Inspecting HTTP requests and responses	7
How does the internet work?	7
HTML Basics	8
Formatting code	10
DOM	11
Validating web pages	11



HTML AND CSS PART-1

Introduction

Welcome to the Ultimate HTML and CSS Part One Series!

In this series, you will learn all the essential skills needed to build fast, beautiful websites that look amazing on any device.

Part One:

- Focuses on the fundamentals of web development, including the basics of HTML and CSS.

Part Two:

- Delves into advanced concepts to enhance your skills further.

Part Three:

- Brings everything together to create a stunning, responsive, and high-performing website.

This course is for

This course is carefully designed for:

- Absolute beginners
- Those with some experience in HTML and CSS want to fill in their knowledge gaps.

If you're already familiar with the basics, feel free to skip to the sections you need. However, I highly encourage you to watch the course from the beginning, as we will be sharing plenty of tips and tricks that you won't want to miss.

Rest assured; this course will not waste your time with unnecessary explanations—we get straight to the point!

Tools You Need for This Course

To start this course, you will need a code editor. There are many code editors available, such as Sublime Text, Visual Studio Code, Notepad++, Vim, Atom, PyCharm, and more. Some of these are paid, while others, like Notepad, are considerably basic.

For this course, we'll be using **Visual Studio Code (VS Code)**, which you can download here:

<https://code.visualstudio.com/>. We will also use a few extensions to enhance our coding experience, such as:

- **Live Server:** To launch your website on a development server.
- **VS Code Icons:** To beautify files icons.
- **Live Preview:** For real-time updates as you code.
- **Prettier - Code Formatter:** To format your code according to best practices automatically.
- **Browser:** Install any browser to run your code.

You're welcome to use your favorite code editor if you are already familiar with it, but make sure it supports the features we'll use.

Additionally, you will need a web browser. Any browser will work, but for this course, we will use **Google Chrome** because of its developer-friendly tools.

Starting & Course Outline

We are about to embark on an exciting web development journey!

- **Starting from the Basics:**
 - We will begin with the fundamentals and take them step by step.
 - By the end of this course, you'll combine all the concepts to create a beautiful web page.
- **HTML and CSS Basics:**
 - Get introduced to the structure and styling of web pages.
 - Learn how to validate web pages—an often-overlooked skill among developers.
- **What You'll Learn in This Section:**
 - **Web Development Fundamentals:**
 - Languages and tools used in web development.
 - Key concepts and terminologies such as:
 - URL, HTTP, DOM, Hosting, Domain, Server, WWW, IP, ISP, DNS, and more.
 - **How the Internet Works:**
 - How websites function behind the scenes.
 - What happens when you use a browser to visit a website.
 - **DevTools for Developers:**
 - Learn how to inspect network traffic using DevTools.
 - Understand how this powerful tool is used by frontend developers.

This journey will be insightful and transformative, equipping you with the skills needed to build impressive web pages. Let's get started! 🚀

Languages and Tools of Web Development

This course is your first step to becoming a web developer. Let's explore a simple roadmap to understand where we are and where we will go next.

Understanding the Basics of Web Development

- **Every website has two main parts:**
 1. **Frontend:**
 - The part you see in a browser, encompassing all the visual aspects.
 2. **Backend:**
 - The part that powers the frontend.
 - It stores data in a database and provides it to the frontend.

Types of Web Development Jobs

Web development jobs are divided into three categories:

- **Frontend Developer:**
 - Focuses on building the visual and interactive elements of a website.
 - Uses **HTML, CSS, and JavaScript** to create the frontend.
- **Backend Developer:**
 - Works on the server side to handle data storage and management.
 - Use various technologies and languages (beyond the scope of this course).
- **Full Stack Developer:**
 - I am a professional who is skilled in both frontend and backend development.

Core Technologies for Frontend Development

1. HTML (Hypertext Markup Language):

- Used to create the structure of a website.
- Think of it as the skeleton of a human body, it provides the framework but isn't visually appealing.

2. CSS (Cascading Style Sheets):

- Used to beautify the structure created with HTML.
- CSS adds colors, layouts, and styles to make the website attractive.

3. JavaScript:

- Adds functionality to a website, like how a human brain enables thinking, talking, and behaving.
- With JavaScript, we can make interactive websites, such as adding forms, animations, or dynamic content.

Why These Languages Are Important

- **HTML, CSS, and JavaScript** are used on nearly every website on the internet.
- The better we learn these languages, the better we will be at building websites.

Front End Roadmap

Your journey to becoming a front-end developer begins here. Let's break it down step by step:

1. Start with HTML and CSS

- Dedicate **4 to 5 hours per day** to learning these foundational languages.
- Within a month, you will have a solid understanding of how to create and style web pages.

2. Learn JavaScript

- Once comfortable with HTML and CSS, move on to JavaScript.
- We have a separate series dedicated to JavaScript to help you get started.
- Learning JavaScript will take approximately **6 weeks**, but mastering it comes with practice and creating projects.

- As you build projects, your skills will grow, and you'll deepen your understanding of JavaScript.

3. Explore Front-End Frameworks and Libraries

- Building websites involves many repetitive tasks. This is where frameworks and libraries come in:
 - They provide pre-written, reusable code to help you complete tasks faster.
 - Popular libraries and frameworks include **React**, **Angular**, and **Vue**.
- Focus on **React** first:
 - React is the most popular library in the front-end space.
 - Once you learn React, you can easily pick up other tools when needed.
 - We also have a separate React series to guide you.

4. Learn Version Control Systems

- Version control systems are essential for:
 - Tracking your project's history.
 - Collaborating with other developers.
- Popular version control tools include:
 - **Git** (the most popular, used by over 70% of development teams).
 - Subversion (SVN), Mercurial, and others.
- Since Git is widely used and appears in nearly every job description, focus on mastering Git first.
- We have a dedicated course for Git to help you get started.

5. Focus on Core Tools and Skills

- The tools and technologies listed above—HTML, CSS, JavaScript, React, and Git—are essential for front-end development.
- These skills are in demand across all job descriptions for front-end developers.
- Remember, different companies use different tools and technologies, and learning never stops.
 - As you move between projects or companies, you may need to learn additional tools, but the foundation remains the same.

Final Tip

Start with these core tools and technologies. Once you've mastered them, expanding your knowledge to include other tools and frameworks will be much easier.

How the web works

Have You Wondered How the Web Works?

Let's learn how the web works step by step:

1. Understanding the URL

- When you type a website name into a browser and hit enter, that name is called a **URL (Uniform Resource Locator)**.

- A URL is a combination of three things:
 - **http** (HyperText Transfer Protocol)
 - **www** (World Wide Web)
 - **Domain Name** (e.g., google.com)
- A URL locates or stores resources on the internet.
- Resources can include:
 - Web pages (HTML documents)
 - Images
 - Videos
 - Audio files
 - PDFs
 - Fonts, and more.

2. The Client-Server Model

- When you hit enter after typing a URL, two main components are involved:
 - **The Browser (Client)** – used by the user to send a request.
 - **The Web Server (Server)** – hosts the website and responds to requests.
- This is called the **Client-Server Model**, where:
 - The client requests a service.
 - The server provides the service.
- Example:

The browser (client) sends a message to the server saying, *"Hey, I need the homepage of the requested website."* This process is like sending a message to a friend, where:

 - The sender = the client.
 - The receiver = the server.

3. HTTP and HTTPS

- These requests are based on a protocol called **HTTP (HyperText Transfer Protocol)**:
 - A textual language used for communication between the client (browser) and server.
 - Not a programming language—just plain text used for communication over the internet.
- **HTTPS (HTTP Secure)**:
 - HTTP with encryption, ensuring secure communication between the client and server.

4. How an HTTP Request Works

- When a browser sends a request, it includes:
 - **Request type** (e.g., GET the file /index.html of the website).
 - **HTTP version** (e.g., HTTP/1.1).

- **Domain location** (e.g., www.google.com).
- **Accepted response type** (e.g., in English).
- The server then processes this request and responds with:
 - A status code (e.g., HTTP/1.1 200 OK):
 - **200 OK** means the request was successful.
 - The date of the response.
 - The file type (e.g., text/html) contains the HTML code.

5. How the Browser Processes the Response

- The browser reads the HTML code and constructs it into a **DOM (Document Object Model)**:
 - The DOM represents the structure of the HTML document.
 - HTML elements include:
 - Document type (<!DOCTYPE html>)
 - HTML, head, body, paragraphs, headings, etc.
- Images, videos, and other resources are stored in different locations.
 - The browser sends separate requests for these resources **in parallel** to ensure the website loads quickly.

6. Rendering the Website

- Once all necessary resources are retrieved, the browser renders (displays) the website on the screen.

Inspecting HTTP requests and responses

- Open the browser and right-click on it.
- Select "Inspect" (also called "DevTools" or "Developer Tools" for short).
- DevTools are used by front-end developers.
- Navigate to the "Network" tab.
- Search for any website, like google.com.
- You will see all the requests and responses, or communication between the client and the server.

Test it yourself by spending 5 minutes exploring the tools. This will help you become familiar with them, and we will learn more about them in the next lectures.

How does the internet work?

The internet is a network that connects billions of devices to each other via wires. It is used to share data between devices, such as my computer and your computer. These devices relate to wire sharing data.

- Some more powerful computers, known as **servers**, are responsible for serving content over the internet. These servers are always online (24/7) and store web pages, serving them to clients.
- The computers we use are referred to as **clients**. A client, or browser, sends requests to access websites. For example, when we enter "google.com" into a browser, the browser sends a request to

the **Internet Service Provider (ISP)**, which is the company responsible for managing the network cables, including those under the sea, that provide internet access.

The ISP relies on **DNS (Domain Name System) servers**. The DNS is responsible for mapping human-readable domain names (like google.com) to IP addresses, which are numerical addresses that are difficult to remember. DNS works similarly to a phone book, where domain names are like contacts saved by their nicknames, and the IP addresses are like phone numbers.

- When we request a website, the DNS server looks up the database to find the exact IP address associated with the domain name.
- Every device connected to the internet has an IP address, which is like the postal code of your computer.

To explore this further, you can use:

- **nslookup.io** to find the IP address of any website.
- **submarinecablemap.com** to visualize the network of cables under the ocean, which includes massive fiber optic cables. Each cable can transmit up to 400 Gbps of data per second.
- **Status code post:**
<http://youtube.com/post/Ugkx61iOHYCKaX7WqqWkxI2wMDfLWPes4LrM?si=fePuQn1fn7FjmnU>

HTML Basics

Creating a Folder and Setting Up HTML in VS Code

1. **Create a Folder:**
 - Right-click in the folder and open it with VS Code (the code editor we just installed).
 - Inside the folder, create a new HTML file named index.html.
2. **Understanding index.html:**
 - The file index.html is commonly referred to as the home page of a website.

Writing HTML Boilerplate Code

Step 1: Declare the Document Type

- The first thing we need to do is tell the browser that this is an HTML document.
- Add `<!DOCTYPE html>` at the top of the file.
 - It represents the version of HTML (HTML5 in this case).
 - HTML is **not case-sensitive**, but it is good practice to use lowercase except for DOCTYPE.

Step 2: Add the Root Element

- The root element of the website is `<html></html>`.
- Inside the `<html>` element, we add two key elements:
 - `<head></head>`
 - `<body></body>`

Structure of the HTML Document

1. <head> Element:

- Represents the head of the website.
- Includes elements like:
 - <title>: Defines the title of the webpage.
 - Meta tags, links, and other non-visible elements.

2. <body> Element:

- Represents the content displayed in the browser's viewport.

3. Example Boilerplate Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>NaseebCodeStudio</title>
</head>
<body>
</body>
</html>
```

4. Relationships:

- <html> is the parent of <head> and <body>.
- <head> and <body> are siblings.

Shortcut for HTML Boilerplate in VS Code

- Use the shortcut: Shift + 1 followed by Enter.
- This will generate the basic boilerplate, including some additional elements we'll learn about later.

Live Server Preview

1. Run the Code in Browser:

- Right-click anywhere in the HTML code and select **Open with Live Server**.
- This will display the result in the browser.

2. Understanding the URL:

- 127.0.0.1: Represents the local computer (IP address).
- 5500: The port where the website is hosted.
- index.html: The file being served.

Adding an Image to the Website

1. Use the Tag:

- The tag is a **self-closing tag** with no child elements.
- It includes two attributes:
 - src: Specifies the source of the image (name and file extension).

- alt: Provides alternative text if the image fails to load.

2. Steps to Add an Image:

- Use the following syntax:
- ``
- Press Ctrl + B to open the file explorer.
- Copy and paste the image into the same directory as index.html.

Introduction to CSS

Adding CSS:

- In the `<head>` element, add a `<style>` tag after the `<title>` element:
- `<style>`
- `</style>`

CSS Basics:

- Target elements using **selectors** (e.g., `img`).
- Add styles using **declarations** enclosed in curly braces `{}`.

Example:

```
<style>
img {
  width: 100px; /* Property: width, Value: 100px */
}
</style>
```

- **Declaration Breakdown:**
 - width: The property.
 - 100px: The value.
 - Use a colon `:` to separate the property and value, and terminate with a semicolon `;`.

Next Steps

- In upcoming lectures, we will:
 - Explore more HTML elements and their exceptions.
 - Learn advanced CSS techniques to style the website further.

Formatting code

We need to write code readable if miss the right alignment or enters in our code we can use an extension called code prettier install from the extension tab in the vs code editor and then right click on the code you will get an option formatting code or press control + shift + p and search for the prettier and then select it the code will be formatted

You can also set the code prettier to format while save go to manage and settings then search for the format on save and then check the check box as you save the file it will auto format.

DOM

DOM or document object

DOM (Document Object Model):

When a browser reads an HTML document, it constructs the document in the form of a DOM. The DOM is a tree-like structure.

To view the DOM:

1. Open a browser and access the Developer Tools.
2. In the "Elements" tab, you will see the DOM.
3. It displays all the elements inside an HTML document.

The DOM is like a tree with a root and leaves, where the leaves represent the elements in HTML. In development, these elements are referred to as nodes in the DOM tree.

The DOM defines the relationships between the elements.

This structure is useful when working with JavaScript, and we will explore it further later in this course.

Validating web pages

While writing code, we may encounter problems. If we don't use the correct syntax, we won't get the expected result.

To identify potential issues in our code, we can use a validator.

For HTML:

- Write your HTML code.
- Google a website called validator.w3.org.
- Upload your file there to check for errors.
- Follow the suggestions provided to fix the mistakes.

For CSS:

- Search for jigsaw.w3.org.

Thanks to this massive infrastructure, we can access websites at lightning speed, often in milliseconds.

The Head Section

The Head Section in HTML

Meta elements in the <head> section provide important information about the webpage. Let's break down the key meta tags:

1. Character Set:

The <meta charset="UTF-8"> tag specifies the character set for the webpage. A character set is a way to represent letters, numbers, and symbols in a format computer can understand. For example:

- Computers don't inherently know what "A" or "B" is. Character sets convert these characters into numbers that computers can process.
- The first widely used character set was **ASCII**, which could only represent basic English characters.

- Now, we use **UTF-8**, which supports almost all characters and symbols from various languages around the world. This ensures your webpage can display text properly in different languages.

2. Viewport Configuration:

The `<meta name="viewport" content="width=device-width, initial-scale=1.0">` tag is crucial for responsive design. Here's what it does:

- The **viewport** is the visible area of a webpage in a browser. It changes depending on the device:
 - On mobile devices, the viewport is smaller.
 - On desktops, you can adjust the viewport by resizing the browser window to see how the website responds.
- This meta tag sets the initial width of the viewport to match the device's screen width and ensures proper zoom levels.
- Including this tag makes your webpage look good on all devices (mobile, tablet, and desktop).

3. Additional Meta Tags:

Here are some other meta tags you can use to provide more information about your webpage:

- **Keywords:**

```
<meta name="keywords" content="HTML, CSS, JavaScript">
```

This was once used for SEO (Search Engine Optimization), but search engines now place less importance on it. Still, you can include it to describe your webpage's content.

- **Description:**

```
<meta name="description" content="NaseebCodeStudio teaches you coding with ease.">
```

This provides a brief description of your webpage. It appears below the website title in search engine results, helping users understand the purpose of the page before clicking.

Why Are Meta Tags Important?

Meta tags help search engines, browsers, and devices to understand your webpage better. At a minimum, every webpage should include these two tags:

1. `<meta charset="UTF-8">`
2. `<meta name="viewport" content="width=device-width, initial-scale=1.0">`

By using these tags effectively, you improve the usability and accessibility of your webpage for a global audience.

Text Elements

Text Elements in HTML

We have learned about the `<p>` (paragraph) element, which is used to display text in paragraph form. Now, let's explore other text elements and their purposes. These elements allow us to emphasize or style specific parts of the content. Here's a breakdown:

Emphasizing Text with `<em>`

- The `<em>` element is used to emphasize content. It represents stress or importance in the text.

Example:

```
<em>This is emphasized text</em>
```

By default, most browsers display emphasized text in *italic*. However, you can style it further using CSS:

```
em {  
    color: red;  
    font-style: normal;  
}
```

- Don't worry about memorizing all this, we'll revisit these concepts multiple times throughout the course.

Italicizing Text with `<i>`

- The `<i>` element is used to italicize content. However, it is now considered outdated and is no longer recommended because HTML is meant for structuring content, not styling it.
- Instead of `<i>`, use CSS to style text as italic when necessary.
- Avoid using `<i>` for purely stylistic purposes.

Highlighting Importance with `<strong>`

- The `<strong>` element is used to signify strong importance, seriousness, or urgency in the text.

Example:

```
<strong>This is a very important text</strong>
```

- By default, most browsers display `<strong>` text in **bold**. Like `<em>`, this can also be styled further using CSS.
- When deciding to use `<strong>`, consider the meaning or context of your content.

Making Text Bold with `<b>`

- In the past, the `<b>` element was used to make text bold. However, it is now considered outdated, similar to `<i>`.
- Since HTML is meant for content structure, avoid using `<b>` for styling purposes.
- Instead, use `<strong>` to indicate importance or CSS to style text as bold when needed.

Summary of Best Practices

- ✓ Use `<em>` for emphasizing text where stress or importance is intended.
- ✓ Use `<strong>` for conveying strong importance or urgency.
- ✓ Avoid using `<i>` and `<b>` for stylistic purposes; rely on CSS for such styling needs.
- ✓ Remember, HTML is for structuring content, while CSS is for styling it.

Wrapping Content in VS Code

If you want to wrap a piece of content or text with an HTML or other element in Visual Studio Code, you can use a quick and easy shortcut. Here's how:

1. **Select the Content:**
Highlight the text or code snippet you want to wrap in an element.
2. **Open the Command Palette:**
Press Ctrl + Shift + P (or Cmd + Shift + P on Mac) to open the Command Palette.
3. **Search for "Wrap with Abbreviation":**
Start typing "**Wrap with Abbreviation**" in the search box and select it when it appears.
4. **Enter the Element Name:**
Type the name of the element you want to use (e.g., div, p, section, etc.) and press Enter.
5. **Done!**
The content will automatically be wrapped in the element you specified.

Why Use This Shortcut?

- It saves time compared to manually creating and placing elements.
- It ensures cleaner and more consistent code formatting.
- You can wrap content in any HTML tag, or even a custom abbreviation using Emmet syntax (e.g., `ul>li*3` for an unordered list with 3 list items).

Additional Tip:

You can also use Emmet abbreviations in this feature to quickly create complex structures. For example, typing `div.container>p` will wrap your content in a div with a class of containers, followed by a p tag inside it.

Heading Elements in HTML

- In HTML, there are six heading elements: `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, and `<h6>`.
- `<h1>` is the most important heading, while `<h6>` is the least important.

Common Mistake:

One common mistake many coders make is using heading elements based on their default size. **This is incorrect** because the appearance of headings (size, color, font, etc.) can always be styled using CSS.

Correct Usage of Headings:

- **Purpose:** Headings should be used to create a logical structure or hierarchy for your webpage's content, not just for visual purposes.
- **Single `<h1>` Rule:** Every webpage should have **only one `<h1>`** element.
 - The `<h1>` represents the main topic or purpose of the page.
 - While having multiple `<h1>` elements on a page won't break your code, it can confuse search engines, making it harder for them to understand what the page is about.
- **Hierarchy:** After `<h1>`, you should use `<h2>` for subtopics, followed by `<h3>`, `<h4>`, and so on. Avoid skipping levels, like jumping directly from `<h1>` to `<h4>`.
 - Think of headings as steps on a staircase: each step should follow logically from the previous one.
- **Multiple Usage:** While `<h1>` should be used only once, other headings (`<h2>` to `<h6>`) can be used multiple times as needed.

Why Proper Heading Structure Matters:

1. **Accessibility:** Properly structured headings make your content easier to navigate for screen readers and assistive technologies.
2. **SEO (Search Engine Optimization):** Search engines rely on headings to understand the content of your page. A well-structured hierarchy improves your page's ranking and visibility.
3. **Readability:** Headings guide users through your content, making it easier to skim and understand.

Example of Proper Heading Usage:

```
<h1>Welcome to My Website</h1> <!-- Main topic of the page -->

<h2>About Us</h2> <!-- Subtopic -->

<p> We are a team of developers dedicated to creating amazing websites. </p>

<h3>Our Mission</h3> <!-- Sub-subtopic -->

<p>Our mission is to make the web a better place. </p>

<h2>Services</h2> <!-- Another subtopic -->

We offer web development, design, and consulting services. </p>

<h3>Web Development</h3> <!-- Sub-subtopic -->

<p>From simple websites to complex applications, we do it all. </p>

<h3>Web Design</h3>

<p>We create stunning and user-friendly designs. </p>
```

By following these best practices, your webpage will be well-organized, accessible, and easier for search engines and users to understand.

HTML Entities

HTML entities are used to display special characters that are reserved in HTML or not easily typed on a keyboard. For example, when you write HTML code, you often need to use angle brackets < > to define tags, but if you want to display those brackets as plain text in a browser, you can't write them directly because the browser interprets them as HTML.

To solve this, we use **HTML entities**. An HTML entity starts with & and ends with ;. Between these characters, we write a specific code that represents the symbol or character.

Common HTML Entities:

1. **Angle Brackets:**

- To display a less-than sign <, use <.
Example: <HTML> will display <HTML>.
- To display a greater-than sign >, use >.

2. Special Symbols:

- Copyright symbol: © → ©
- Registered trademark: ® → ®
- Non-breaking space:

3. Emojis:

- 😅 → 😊
- 🚀 → 🚀

What is Non-Breaking Space?

A **non-breaking space** () is used to add space between words or characters without allowing the browser to break the line at that point. This is helpful in keeping certain words together on the same line. For example:

Hello World! → Hello World! (but "Hello" and "World!" will stay on the same line).

More HTML Entities

There are many HTML entities available, but you don't need to memorize all of them because most are rarely used. You can find a full list at this website:

[HTML Entity Reference](#)

Hyperlinks

Hyperlinks are essential for navigation between web pages or websites. They are created using the <a> (anchor) element in HTML.

1. Basic Syntax

The basic syntax of an anchor element is:

```
<a href="URL">Clickable Text or Element</a>
```

- **href attribute:** This stands for "Hypertext Reference" and specifies the link's target URL.

2. Types of URLs

- **Relative URL:**
A relative URL is based on the current page's location.

If the target page is in the same folder

```
<a href="about.html">About Page</a>
```

If the target page is in a different folder:

```
<a href="folderName/about.html">About Page</a>
```

To navigate one folder up (parent folder), use ../:

```
<a href="../index.html">Home Page</a>
```


For deeply nested folders, use multiple ../:

```
<a href="../../index.html">Home Page</a>
```

- **Absolute URL:**

An absolute URL specifies the full path to a resource, starting from the root of the website or including the protocol (e.g., https://).

Example:

```
<a href="/index.html">Home Page</a>
```

```
<a href="https://google.com">Google</a>
```

3. Linking to Files

You can link to other file types like PDFs, images, documents, etc.:

```
<a href="document.pdf">Download PDF</a>
```

4. Prompting Downloads

To allow users to download files directly, use the download attribute:

```
<a href="image.png" download>Download Image</a>
```

5. Linking Within the Same Page (Fragments)

You can link to specific sections of a page using **fragments**.

1. Add an id to the target element:

```
<h2 id="CSS-Section">CSS Section</h2>
```

2. Link to that id:

```
<a href="#CSS-Section">Read CSS</a>
```

To create a "Jump to the Top" link

```
<a href="#">Jump to Top</a>
```

6. Opening Links in a New Tab

To open a link in a new tab, use the target="_blank" attribute

```
<a href="https://google.com" target="_blank">Open Google</a>
```

7. Linking Emails

You can link to an email address using mailto::

```
<a href="mailto:naseebkhan.contact.me@gmail.com">Email Me</a>
```

8. Difference Between a Link and a Hyperlink

- A **link** is just the address or URL of a page (e.g., https://google.com).
- A **hyperlink** is the clickable element (text, image, etc.) that takes the user to the linked address.

9. Best Practices

- Use descriptive text for links to improve accessibility (e.g., "Read More About CSS" instead of "Click Here").

- Use relative URLs for internal links and absolute URLs for external websites.
- Use fragments and ids to improve page navigation.
- Open external links in a new tab to keep users on your website.

